ECE551: Digital System Design and Synthesis

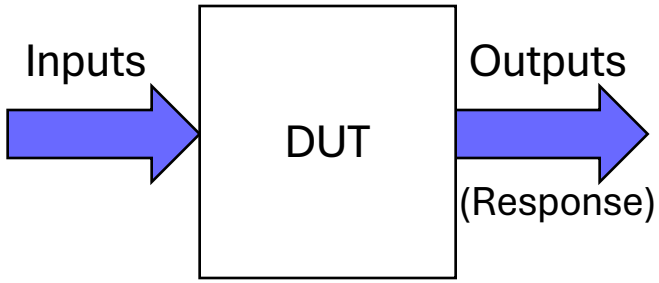
Fall 2026

Testbench Intro

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Testbench Basics

- Need to verify design
  - Check how design behaves under different inputs

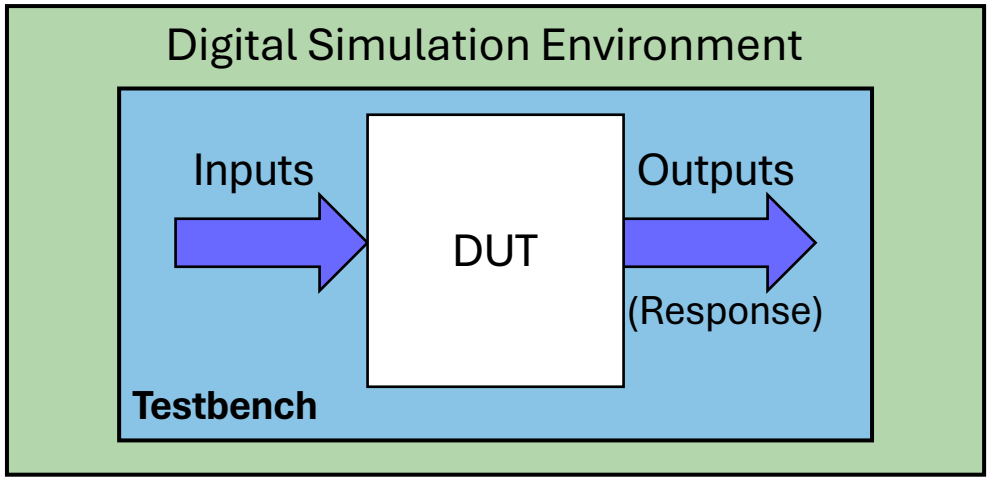


- Need to figure out **what** to test (testcases)
  - What inputs do I need
  - What output do I expect (and when)

- **How** to test?
  - Digital simulation using testbenches

Testbench Basics

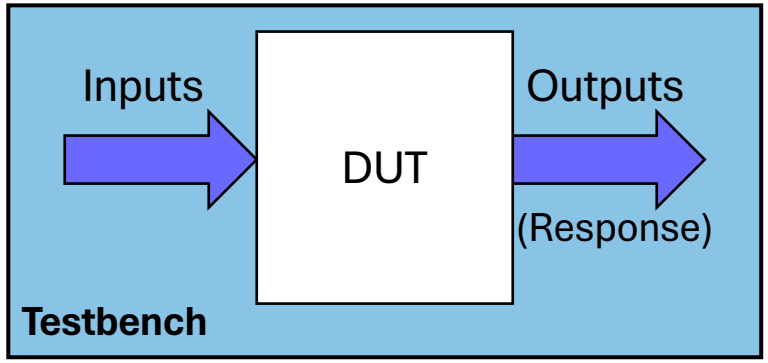
- Need to verify design
  - Check how design behaves under different inputs



- Need to figure out **what** to test (testcases)
  - What inputs do I need
  - What output do I expect (and when)
- **How** to test?
  - Digital simulation using testbenches

Testbench Basics

- A module with no ports
- Minimum Requirement:
  - Instantiate the unit being tested (DUT)
  - Provide input to that unit
    - Usually a few different combinations
- Watch the outputs
  - Human…
    - Subject to errors
    - Cannot be automated
  - Self-checks



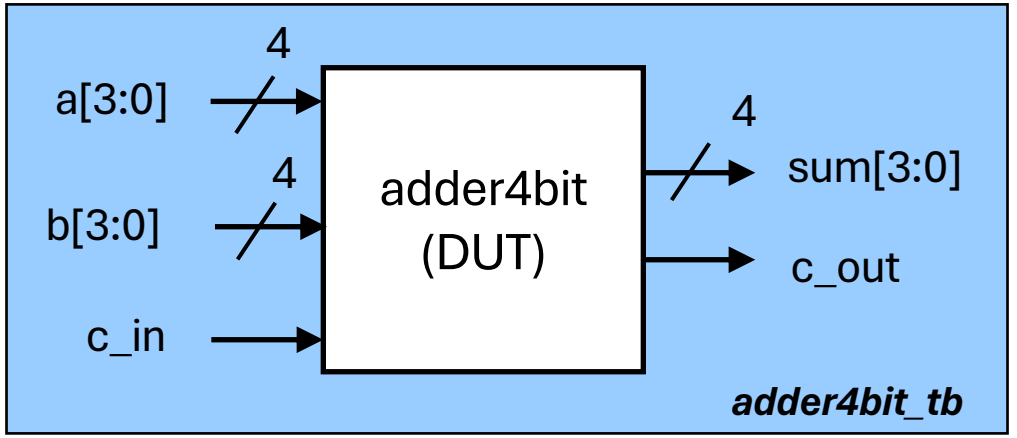
Testbench Example with Human Monitoring

```
`timescale 1ns /10 ps // time_unit/time_precision

module adder4bit_tb;  // no ports
  reg[8:0] stim;      // input stimulus to DUT are regs
  wire[3:0] S;        // outputs of DUT are wires
  wire C4;

  // instantiate DUT
  adder4bit(.sum(S), .c_out(C4), .a(stim[8:5]), .b(stim[4:1]), .c(stim[0]));

  // stimulus generation
  initial begin
    stim = 9'b0000_0000_0;      // at 0 ns
    #10 stim = 9'b1111_0000_1;  // at 10 ns
    #10 stim = 9'b0000_1111_1;  // at 20 ns
    #10 stim = 9'b1111_0001_0;  // at 30 ns
    #10 stim = 9'b0001_1111_0;  // at 40 ns
    #10 $stop;                  // at 50 ns - stops simulation
  end
endmodule
```



Manually check waveforms

Output Test Info

- Several different system tasks to output info
  - **$display**, **$strobe**
    - Output specific information like a `printf` in a C program
  - **$monitor**
    - Output the given values whenever one changes

Simulation only. Ignored by synthesis tools.

- Can use format strings
  - %h, %H hex
  - %d, %D decimal
  - %o, %O octal
  - %b, %B binary
  - %t time

```
$monitor("%t: %b %h %h %h %b\n", $time, c_out, sum, a, b, c_in);
```

Testbench Example

```
`timescale 1ns /10 ps // time_unit/time_precision

module adder4bit_tb;  // no ports
  reg[8:0] stim;       // input stimulus to DUT are regs
  wire[3:0] S;         // outputs of DUT are wires
  wire C4;

  // instantiate DUT
  adder4bit(.sum(S), .c_out(C4), .a(stim[8:5]), .b(stim[4:1]), .c(stim[0]));

  initial $monitor("%t A:%h B:%h ci:%b Sum:%h co:%b\n", $time,stim[8:5],stim[4:1],stim[0],C4,S);

  // stimulus generation
  initial begin
    stim = 9'b0000_0000_0;       // at 0 ns
    #10 stim = 9'b1111_0000_1;   // at 10 ns
    #10 stim = 9'b0000_1111_1;   // at 20 ns
    #10 stim = 9'b1111_0001_0;   // at 30 ns
    #10 stim = 9'b0001_1111_0;   // at 40 ns
    #10 $stop;                   // at 50 ns – stops simulation
  end
endmodule
```

Simulation Control System Tasks

- $stop
  - Stops simulation and allows you to still probe signals and debug
- $finish
  - Completely stops simulation, simulator relinquishes control of thread